

Лабораторная работа № 5

Решение проблемы собственных значений и собственных векторов

Точные методы

Метод Леверрье

Procedure Leverre(Mat: TMatr);

(в процедуре используются следующие процедуры и функции: *Copу* – копирование матрицы; *MatrUmn* – перемножение матриц; *Resh* – нахождение решения характеристического многочлена методом Ньютона; *SobVect* – нахождение собственных векторов матрицы).

В х о д н ы е п а р а м е т р ы : *Matr: TMatr* – исходная матрица (формируется перед запуском).

В ы х о д н ы е п а р а м е т р ы : вычисляются и выводятся на экран собственные значения и вектора.

Схема алгоритма приведена на рисунке 22.

Пример: Вычислить собственные значения и собственные вектора матрицы

$$\begin{pmatrix} 1 & -1 & -1 & 2 \\ 2 & 3 & 0 & -4 \\ 1 & 1 & -2 & -2 \\ 1 & 1 & 0 & -1 \end{pmatrix}$$

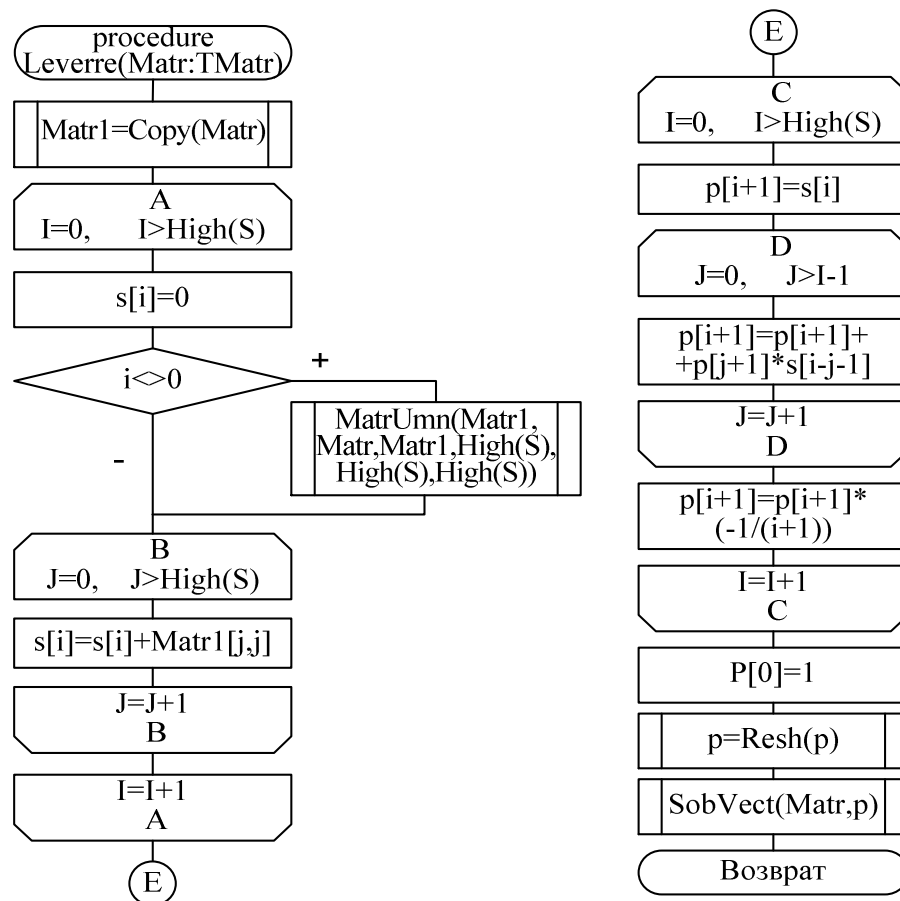
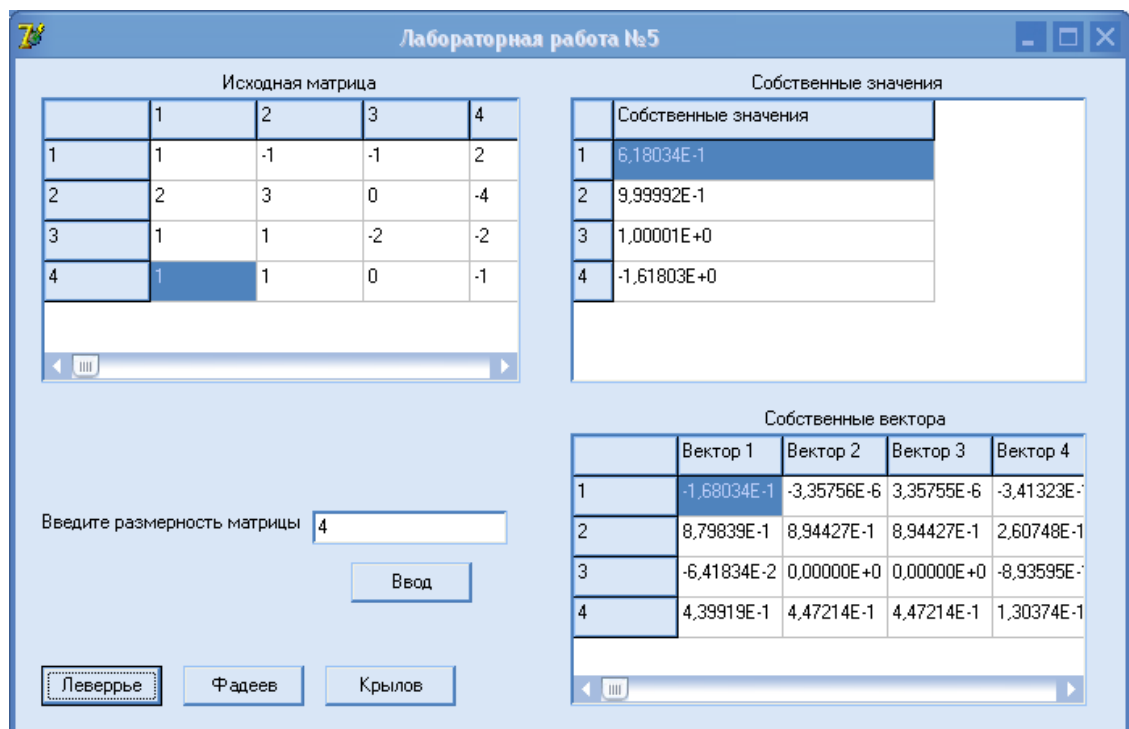


Рисунок 22 – Схема алгоритма метода Леверрье

Вычисления по программе привели к следующим результатам



Варианты заданий для нахождения собственных значений и собственных векторов матрицы приведены в таблице 5.

Метод Фадеева

Procedure Fadeev(Mat: TMatr);

(в процедуре используются следующие процедуры и функции: *Copу1* – копирование матрицы; *MatrUmn* – перемножение матриц; *Resh* – нахождение решения характеристического многочлена методом Ньютона; *SobVect* – нахождение собственных векторов матрицы).

В х о д н ы е п а р а м е т р ы : *Matr: TMatr* – исходная матрица (формируется перед запуском).

В ы х о д н ы е п а р а м е т р ы : находятся и выводятся на экран собственные значения и вектора.

Схема алгоритма приведена на рисунке 23.

Пример. Вычислить собственные значения и собственные вектора матрицы

$$\begin{pmatrix} 1 & -1 & -1 & 2 \\ 2 & 3 & 0 & -4 \\ 1 & 1 & -2 & -2 \\ 1 & 1 & 0 & -1 \end{pmatrix}$$

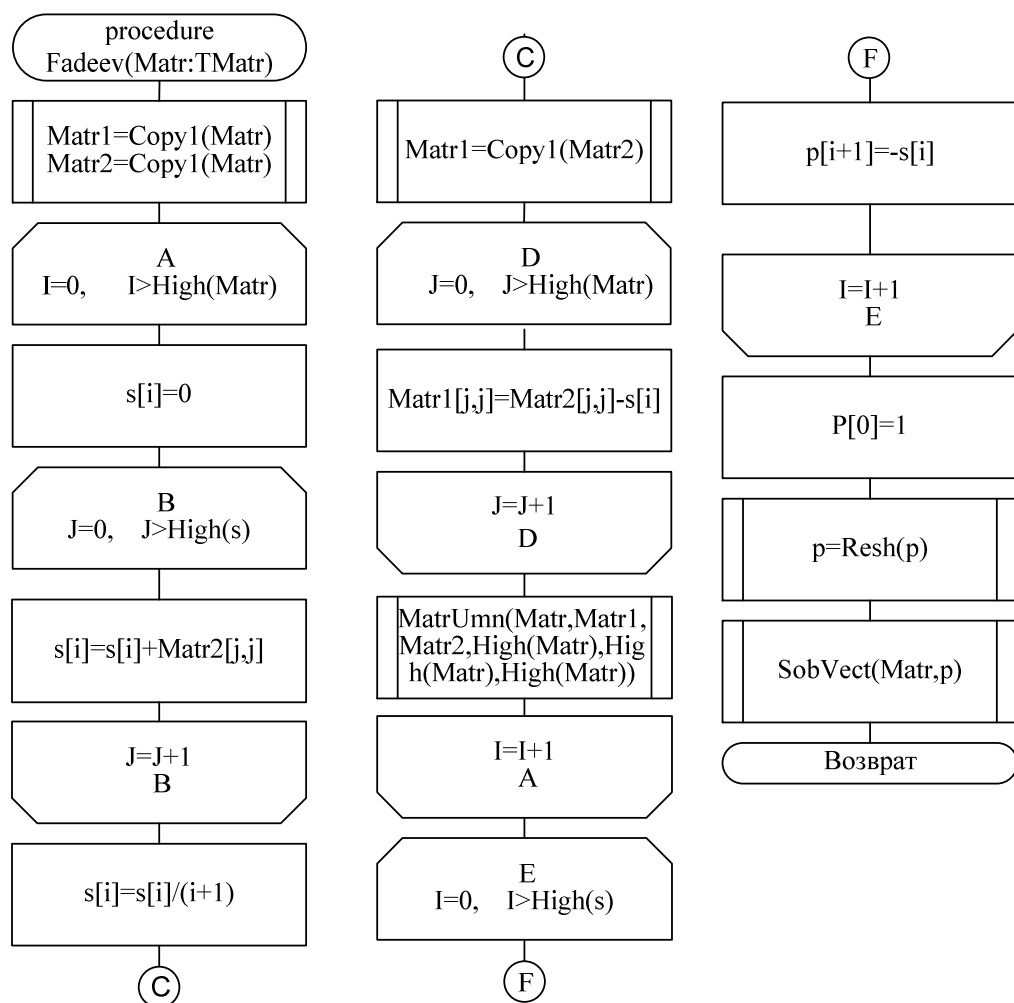
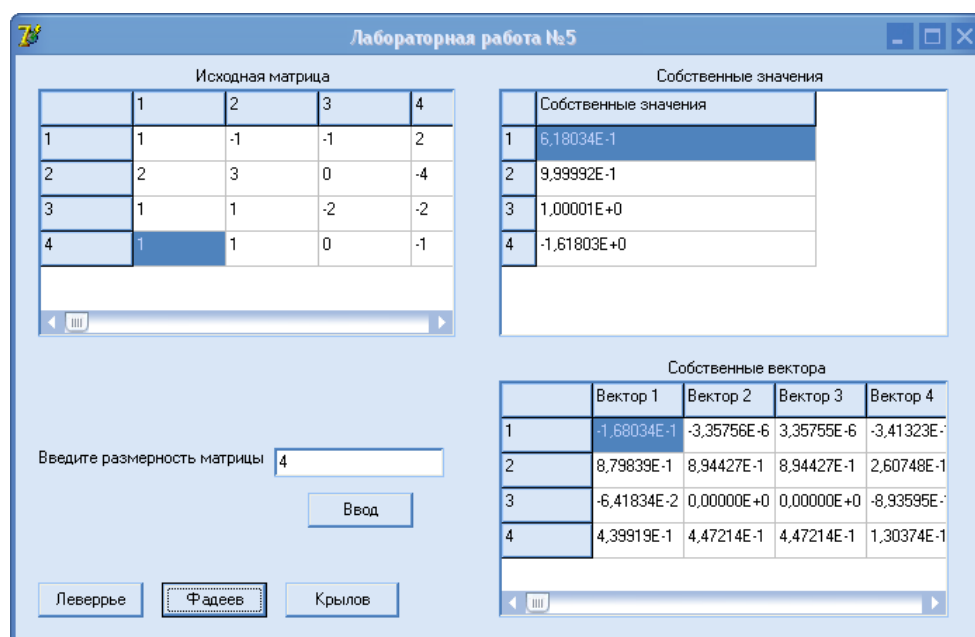


Рисунок 23 – Схема алгоритма метода Фадеева

Вычисления по программе привели к следующим результатам



Варианты заданий для нахождения собственных значений и собственных векторов матрицы приведены в таблице 5.

Метод Крылова

Procedure Krilov(Mat:TMatr);

(в процедуре используются следующие процедуры и функции: *Copy* – копирование матрицы; *MatrUmnMas* – умножение матрицы на массив; *CopyMas* – копирование массива; *CopyTrans* – копирование и транспонирование матрицы; *Det* – нахождение определителя матрицы; *SIMQ* – решение системы линейных алгебраических уравнений. (Процедура берется из первой лабораторной). *Resh* – нахождение решения характеристического многочлена методом Ньютона; *SobVect* – нахождение собственных векторов матрицы).

В х о д н ы е п а р а м е т р ы : *Mat: TMatr* – исходная матрица (формируется перед запуском).

В ы х о д н ы е п а р а м е т р ы : находятся и выводятся на экран собственные значения и вектора.

Схема алгоритма приведена на рисунке 24.

Пример. Вычислить собственные значения и собственные вектора матрицы

$$\begin{pmatrix} 1 & -1 & -1 & 2 \\ 2 & 3 & 0 & -4 \\ 1 & 1 & -2 & -2 \\ 1 & 1 & 0 & -1 \end{pmatrix}$$

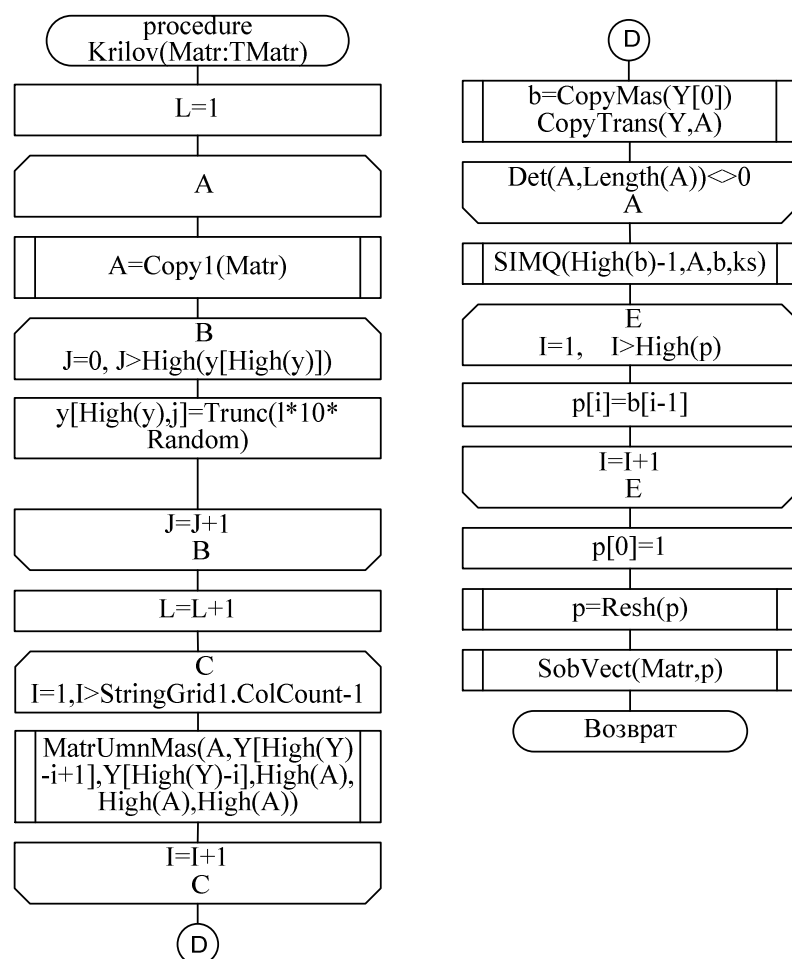
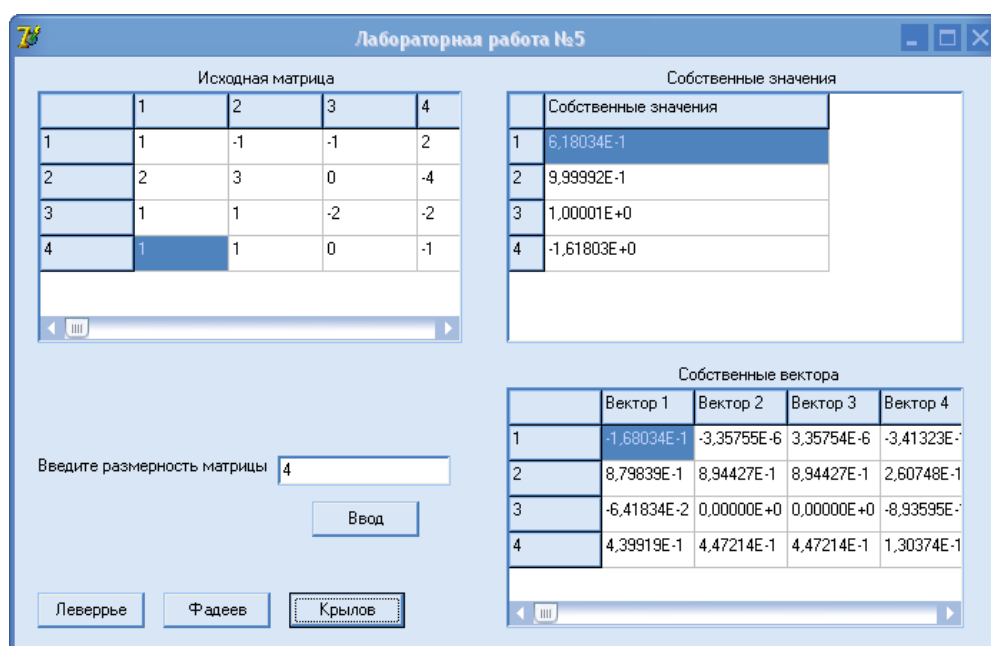


Рисунок 24 – Схема алгоритма метода Крылова

Вычисления по программе привели к следующим результатам



Варианты заданий для нахождения собственных значений и собственных векторов матрицы приведены в таблице 5.

Текст программы вычисления собственных значений и векторов методами: Леверрье, Фадеева, Крылова:

unit Unit1; // программа вычисления собственных значений и векторов методами: Леверрье, Фадеева, Крылова.

interface

uses

Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls, Forms,
Dialogs, StdCtrls, Grids;

type

TForm1 = class(TForm)
StringGrid1: TStringGrid;
Button1: TButton;
Edit1: TEdit;
StringGrid2: TStringGrid;
Button2: TButton;
StringGrid3: TStringGrid;
Button6: TButton;
Label1: TLabel;
Button3: TButton;
Label2: TLabel;
Label3: TLabel;
Label4: TLabel;
procedure Button2Click(Sender: TObject);
procedure Button1Click(Sender: TObject);
procedure Button6Click(Sender: TObject);
procedure Button3Click(Sender: TObject);

private

{ Private declarations }

public

{ Public declarations }

end;

var

Form1: TForm1;

implementation

{ \$R *.dfm }

type

TMas=array of real;
TMatr=array of TMas;
TPer=array of Integer;

function Vkl(Per:TPer;Znach,Kol:integer):Boolean; //проверяет входит ли в массив значение Znach, используется при вычислении определителя

```
var
  i:integer;
begin
  result:=false;
  for i:=0 to kol-1 do
    if Per[i]=Znach then
      begin
        result:=true; exit;
      end;
  end;
```

function Perestановка(Per:TPer;n:Integer):boolean;//для определителя указывает знак с каким входит в сумму очередное слагаемое

```
var
  i,j,kol:integer;
begin
  kol:=0;
  for i:=0 to n-2 do
    for j:=i+1 to n-1 do
      if Per[i]>Per[j] then
        inc(kol);
  result:= ODD(kol);
end;
```

function SumMatrToPer(Matr:TMatr;Per:TPer;n:Integer):Extended;// формирует очередное слагаемое в определителе

```
var
  i:integer;
begin
  result:=1;
  for i:=0 to n-1 do
    result:=result*Matr[i,Per[i]];
  if Perestановка(Per,n) then
    result:=-1*result;
end;
```

function DetRec(Matr:TMatr;const n:integer;Per:TPER;n0:integer):Extended;
//рекурсивно формирует перестановки и ищет определитель

```
var
  i:integer;
begin
  result:=0;
  for i:=0 to n-1 do
    begin
      if Vkl(Per,i,n0) then
        continue
      else
        begin
          Per[n0]:=i;
```

```

    if n0=n-1 then
    begin
        result:=SumMatrToPer(Matr,Per,n);
    end
    else
        result:=result+DetRec(Matr,n,Per,n0+1);
    end;
end;
end;
end;

```

function Det(Matr:TMatr;n:Integer):Extended;// подготавливает массив и запускает ре-
курсию для нахождения определителя

```

var
    Per:TPer;
begin
    SetLength(Per,n);
    Per[0]:=1;
    result:=DetRec(Matr,n,Per,0);
end;

```

Function SQRN(a:real;b:integer):double;//возводит в степень В число А

```

begin
    if a>0 then
        sqrn:=exp(b*ln(a))
    else
        if a<>0 then
            begin
                if odd(b)=true then
                    begin
                        sqrn:=-exp(b*ln(abs(a)));
                    end
                else
                    sqrn:=exp(b*ln(abs(a)));
                end
            end
        else
            sqrn:=0;
        end;
end;

```

procedure MatrUmn(a,b:TMatr;var c:TMatr;n,m,k:integer);//перемножение матриц

```

var
    i,j,l:integer;
    s:real;
begin
    SetLength(c,n+1,k+1);
    for i:=0 to n do
        begin
            for j:=0 to k do
                begin
                    s:=0;
                    for l:=0 to m do
                        s:=s+a[i,l]*b[l,j];
                    c[i,j]:=s;
                end
            end
        end
    end;

```

```

    end;
    end;
end;

function Proizv(xar:TMas):TMas;// считает производную многочлена, переданного в
массиве
var
  i:integer;
  proizv:TMas;
begin
  SetLength(proizv,High(xar));
  for i:=0 to High(xar)-2 do
    proizv[i]:=xar[i]*(High(xar)-i);
  proizv[High(xar)-1]:=xar[High(xar)-1];
  result:=proizv;
end;

Function Delenie(f:TMas;koren:real):TMas;// делит многочлен на одночлен (корень),
тем самым уменьшая его степень
var
  i:integer;
  otv:TMas;
begin
  SetLength(otv,High(f)+1);
  otv[0]:=f[0];
  for i:=1 to High(f) do
    otv[i]:=(koren*otv[i-1])+f[i];
  result:=otv;
end;

function Podstanovka(xar:TMas;kor:real):real;//подставляет число в многочлен
var
  i:integer;
  otv:real;
begin
  otv:=0;
  for i:=0 to High(xar)-1 do
    begin
      otv:=otv+SQRN(kor,High(xar)-i)*xar[i];
    end;
  otv:=otv+xar[High(xar)];
  result:=otv;
end;

function Resh(xar:TMas):TMas;//находит решение многочлена
var
  dx,xn,xn1:real;
  i,p:integer;
  f1,otv:TMas;
begin
  SetLength(otv,High(xar));
  p:=High(xar);

```

```

for i:=1 to p do
begin
  xn:=0.00001;
  f1:=(Proizv(xar));
  repeat
    dx:=(Podstanovka(xar,xn))/(Podstanovka(f1,xn));
    xn1:=dx+xn;
    xn:=xn1;
  until (abs(dx)<0.00001)or(Podstanovka(xar,xn1)=0);
  xar:=Delenie(xar,xn1);
  SetLength(xar,High(Xar));
  otv[i-1]:=xn1;
  Form1.StringGrid2.Cells[1,i]:=FloatToStrF(xn1,ffExponent,6,13);
end;
result:=otv;
end;

```

```

procedure TForm1.Button2Click(Sender: TObject);//подготавливает к вводу StringGrid-

```

ы

```

var
  i:integer;
begin
  StringGrid1.ColCount:=StrToInt(Edit1.Text)+1;
  StringGrid1.RowCount:=StrToInt(Edit1.Text)+1;
  StringGrid2.RowCount:=StrToInt(Edit1.Text)+1;
  StringGrid3.RowCount:=StrToInt(Edit1.Text)+1;
  StringGrid3.ColCount:=StrToInt(Edit1.Text)+1;
  StringGrid2.Cells[1,0]:='Собственные значения';
  for i:=1 to StrToInt(Edit1.Text) do
  begin
    StringGrid1.Cells[0,i]:=IntToStr(i);
    StringGrid3.Cells[0,i]:=IntToStr(i);
    StringGrid2.Cells[0,i]:=IntToStr(i);
    StringGrid1.Cells[i,0]:=IntToStr(i);
    StringGrid3.Cells[i,0]:='Вектор '+IntToStr(i);
  end;
end;

```

Function sum(Matr:TMatr;Mas:TMas;p:integer):real; //находит значение очередного неизвестного, считая сумму последующих элементов и деля её на элемент на главной диагонали

```

var
  i:integer;
  sun:real;
begin
  sun:=0;
  for i:=p+1 to High(Matr) do
    sun:=sun+Matr[p,i]*Mas[i];
  sun:=-sun/Matr[p,p];
  result:=sun;
end;

```

```

function Perest(Var Matr:TMatr;p,i:integer):boolean;
var
  u,l:integer;
  rec:real;
begin
  result:=false;
  for u:=p+1 to High(Matr) do
    if Matr[u,i]<>0 then
      begin
        for l:=0 to High(Matr) do
          begin
            rec:=Matr[p,l];
            Matr[p,l]:=Matr[u,l];
            Matr[u,l]:=rec;
          end;
        result:=true;
        break;
      end;
  end;

procedure Minim(var Matr:TMatr);//заменяет все элементы в матрице меньше 0.0001 на 0
var
  i,j:integer;
begin
  for i:=0 to High(Matr) do
    for j:=0 to High(Matr) do
      if Abs(Matr[i,j])<0.0001 then
        Matr[i,j]:=0
    end;

  procedure Prov(Matr:TMatr;var b1:TMas;k,l:integer);// делается проверка, если решение
до этого было выбрано любое, а теперь выясняется что оно не подходит, то оно заменяет-
ся 0
  var
    i:integer;
  begin
    for i:=l+1 to High(Matr) do
      if Matr[k,i]<>0 then
        b1[i]:=0
    end;

  function Stup(var Matr:TMatr):TMas;//приводим матрицу к ступенчатому виду и нахо-
дим любое частное решение
  var
    k,i,j:integer;
    b:real;
    b1:TMas;
  begin
    for i:=0 to High(Matr)-1 do
      begin
        for k:=i+1 to High(matr) do
          begin

```

```

    if abs(Matr[i,i])=0 then //break;
    if Perest(Matr,i,i)<>true then break;
    b:=-matr[k,i]/Matr[i,i];
    for j:=0 to High(Matr) do
    begin
        Matr[k,j]:=Matr[i,j]*b+Matr[k,j];
    end;
    Minim(Matr);
end;
end;
SetLength(b1,High(Matr)+1);
for i:=High(Matr) downto 0 do
if abs(Matr[i,i])=0 then
begin
    b1[i]:=1;
    Prov(Matr,b1,i,i);
end
else
    b1[i]:=sum(Matr,b1,i);
result:=b1
end;

```

```

Function Copy1(Matr:TMatr):TMatr;//копируем матрицу
var
    i,j:integer;
    Matr1:TMatr;
begin
    SetLength(Matr1,High(Matr)+1,High(Matr)+1);
    for i:=0 to High(Matr) do
        for j:=0 to High(Matr) do
            Matr1[i,j]:=Matr[i,j];
        result:=Matr1;
    end;

```

```

function CopyMas(const Mas:TMas):TMas;//копируем массив
var
    i:integer;
begin
    SetLength(result,Length(Mas)+1);
    for i:=0 to High(Mas) do
        result[i]:=Mas[i];
    end;

```

```

function Clear1(Mas:TMas):TMas;//очищаем массив
var
    i:integer;
    Mas1:TMas;
begin
    SetLength(Mas1,High(Mas)+1);
    for i:=0 to High(Mas) do
        Mas1[i]:=0;
    result:=Mas1;

```

```

end;

procedure OutPut(otv1:TMas;p:integer);//нормализуем массив
var
  i:integer;
  s:real;
begin
  s:=0;
  for i:=0 to High(otv1) do
    s:=s+SQR(otv1[i]);
  s:=SQRT(s);
  for i:=0 to High(otv1) do
    begin
      otv1[i]:=otv1[i]/s;
      Form1.StringGrid3.Cells[p,i+1]:=FloatToStrF(otv1[i],ffExponent,6,13);
    end;
  end;
end;

procedure SobVect(Mat:TMatr;otv:TMas); //находим собственные вектора для собст-
венных значений
var
  i,k:integer;
  Matr1:TMatr;
  otv1:TMas;
begin
  SetLength(otv1,High(Mat)+1);
  for k:=0 to High(otv)do
    begin
      Matr1:=Copy1(Mat);
      for i:=0 to High(otv) do
        begin
          Matr1[i,i]:=Mat[i,i]-otv[k];
        end;
      Minim(Matr1);
      otv1:=Clear1(otv1);
      otv1:=Stup(Matr1);
      OutPut(otv1,k+1);
    end;
  end;
end;

procedure Leverre(Mat:TMatr);
var
  i,j:integer;
  Matr1:TMatr;
  S,P:TMas;
begin
  SetLength(S,Form1.StringGrid1.ColCount-1);
  SetLength(P,Form1.StringGrid1.ColCount);
  Matr1:=Copy(Mat);
  For i:=0 to High(S) do
    begin
      s[i]:=0;

```

```

if i<>0 then
  MatrUmn(Matr1,Matr,Matr1,High(S),High(S),High(S));
for j:=0 to High(s) do
  s[i]:=s[i]+Matr1[j,j];
end;
for i:=0 to High(s) do
begin
  p[i+1]:=s[i];
  for j:=0 to i-1 do
    begin
      p[i+1]:=p[i+1]+p[j+1]*s[i-j-1];
    end;
  p[i+1]:=p[i+1]*(-1/(i+1));
end;
p[0]:=1;
p:=Resh(p);
SobVect(Matr,p);
end;

```

procedure TForm1.Button1Click(Sender: TObject);//находим коэффициенты характеристического многочлена методом Лаверрье

```

var
  Matr:TMatr;
  i,j:integer;
begin
  SetLength(Matr,StringGrid1.ColCount-1,StringGrid1.ColCount-1);
  for i:=1 to StringGrid1.ColCount-1 do
    for j:=1 to StringGrid1.RowCount-1 do
      begin
        Matr[i-1,j-1]:=StrToFloat(StringGrid1.Cells[j,i]);
      end;
  Leverre(Matr);
end;

```

```

procedure Fadeev(Matr:TMatr);
var
  i,j:integer;
  Matr1,Matr2:TMatr;
  S,P:TMas;
begin
  SetLength(S,Form1.StringGrid1.ColCount-1);
  SetLength(P,Form1.StringGrid1.ColCount);
  Matr1:=Copy1(Matr);
  Matr2:=Copy1(Matr);
  For i:=0 to High(Matr) do
    begin
      s[i]:=0;
      for j:=0 to High(s) do
        s[i]:=s[i]+Matr2[j,j];
      s[i]:=s[i]/(i+1);
      Matr1:=Copy1(Matr2);
      for j:=0 to High(Matr) do

```

```

    Matr1[j,j]:=Matr2[j,j]-s[i];
    MatrUmn(Matr,Matr1,Matr2,High(Matr),High(Matr),High(Matr));
end;
for i:=0 to High(s) do
    p[i+1]:=-s[i];
    p[0]:=1;
    p:=Resh(p);
    SobVect(Matr,p);
end;

procedure TForm1.Button6Click(Sender: TObject);//находим коэффициенты характери-
стического многочлена методом Фадеева
var
    i,j:integer;
    Matr:TMatr;
begin
    SetLength(Matr,StringGrid1.ColCount-1,StringGrid1.ColCount-1);
    for i:=1 to StringGrid1.ColCount-1 do
        for j:=1 to StringGrid1.RowCount-1 do
            begin
                Matr[i-1,j-1]:=StrToFloat(StringGrid1.Cells[j,i]);
            end;
        Fadeev(Matr);
    end;
end;

procedure MatrUmnMas(a:tmatr;b:tmas;Var c:tmas;n,m,k:byte);// находим произведение
матрицы на массив
var
    s:real;
    i,j,l:byte;
Begin
    for i:=0 to n do
        begin
            for j:=0 to k do
                begin
                    s:=0;
                    for l:=0 to m do
                        s:=s+a[i,l]*b[l];
                    c[i]:=s;
                end;
            end;
        end;
    end;

procedure CopyTrans(A:TMatr;var OutPut:TMatr);//копируем и транспонируем матри-
цу, удаляя последнюю строчку
var
    i,j:integer;
begin
    for i:=0 to High(A)-1 do
        for j:=0 to High(a)-1 do
            OutPut[i,j]:=a[j+1,i];
        end;
    end;
end;

```

```

procedure SIMQ(n:integer; a:tmatr; var b:tmas;var ks:Integer);
label m1;
const eps=1e-21;
var
  max,u,v:real;
  i,j,k1,l:integer;
begin
  for i:=0 to n do a[i,n+1]:=-b[i];
  for i:=0 to n do
    begin
      max:=abs(a[i,i]); k1:=i;
      for l:=i+1 to n do if (abs(a[l,i])>max) then
        begin
          max:=abs(a[l,i]); k1:=l;
        end;
      if(max<eps)then begin ks:=1; goto m1;
      end else ks:=0;
      if k1<>i then
        for j:=i to n+1 do
          begin u:=a[i,j]; a[i,j]:=a[k1,j]; a[k1,j]:=u; end;
          v:=a[i,i];
          for j:=i to n+1 do a[i,j]:=a[i,j]/v;
          for l:=i+1 to n do begin
            v:=a[l,i]; for j:=i+1 to n+1 do a[l,j]:=a[l,j]-a[i,j]*v;
          end; end;
          b[n]:=a[n,n+1];
          for i:=n-1 downto 0 do begin b[i]:=a[i,n+1];
            for j:=i+1 to n do b[i]:=b[i]-a[i,j]*b[j];
          end;
        m1:end;

procedure Krilov(Mat:TMatr);
var
  A,Y:TMatr;
  b,p:TMas;
  ks,i,j,l:integer;
begin
  SetLength(p,Form1.StringGrid1.ColCount);
  SetLength(A,form1.StringGrid1.ColCount-1,Form1.StringGrid1.ColCount-1);
  SetLength(Y,Form1.StringGrid1.ColCount,Form1.StringGrid1.ColCount-1);
  SetLength(b,Form1.StringGrid1.ColCount-1);
  l:=1;
  repeat
    A:=Copy1(Mat);
    for j:=0 to High(y[High(y)]) do
      y[High(y),j]:=Trunc(l*10*Random);
    inc(l);
    for i:=1 to Form1.StringGrid1.ColCount-1 do
      MatrUmnMas(A,Y[High(Y)-i+1],Y[High(Y)-i],High(A),High(A),High(A));
    b:=CopyMas(Y[0]);
    CopyTrans(Y,A);

```

```

until Det(A,Length(A))<>0 ;
SetLength(A,Form1.StringGrid1.ColCount-1,Form1.StringGrid1.ColCount);
SIMQ(High(b)-1,A,b,ks);
for i:=1 to High(p) do
  p[i]:=b[i-1];
p[0]:=1;
p:=Resh(p);
SobVect(matr,p);
end;

procedure TForm1.Button3Click(Sender: TObject);//находим коэффициенты характери-
стического многочлена методом Крылова
var
  Matr:TMatr;
  i,j:integer;
begin
  SetLength(Matr,StringGrid1.ColCount-1,StringGrid1.ColCount-1);
  for i:=1 to StringGrid1.ColCount-1 do
    for j:=1 to StringGrid1.RowCount-1 do
      begin
        Matr[i-1,j-1]:=StrToFloat(StringGrid1.Cells[j,i]);
      end;
    Krilov(Matr);
  end;
end.

```